



The AI Foundation

Field Notebook

12 chapters of analogies, diagrams & memory hooks — built so you remember things, not just read them.

PREPARED FOR · AI ENGINEER CERTIFICATION · CHENNAI

What's inside

01	The AI Family Tree	MED	02	Building Blocks of GenAI	HIGH
03	Prompt Engineering Essentials	HIGH	04	Prompt Patterns & Practice	HIGH
05	RAG — Retrieval Augmented Generation	HIGH	06	LangChain	HIGH
07	Foundations of Agentic AI	HIGH	08	Agent Types, Lifecycle & Architecture	HIGH
09	The Agentic AI Loop	HIGH	10	Multi-Agent Systems	MED
11	Agentic Workflow & Tech Stack	MED	12	Challenges, Ethics & Future	MED

01

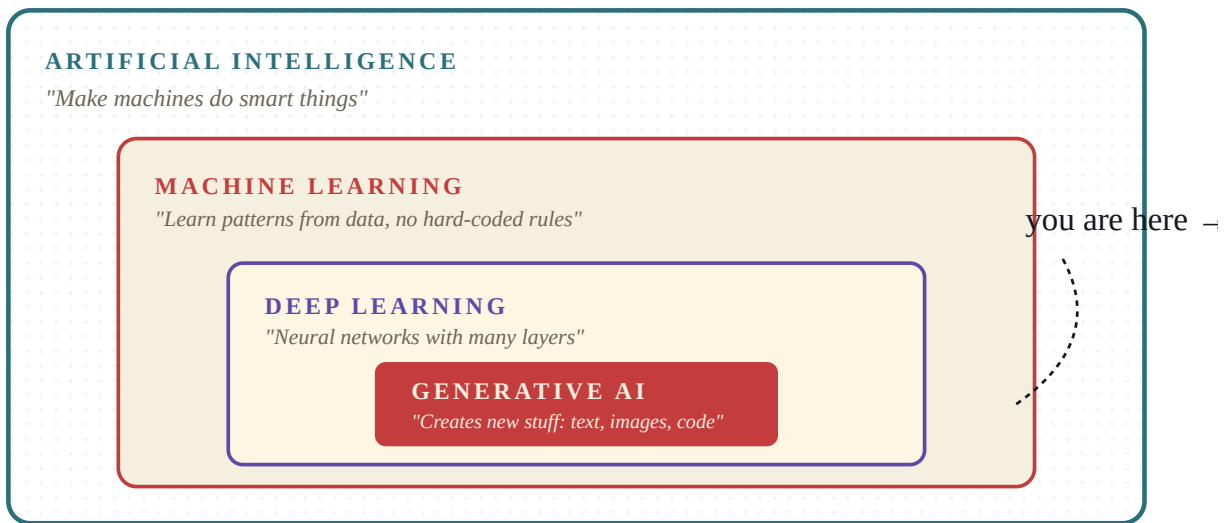
CHAPTER ONE • FOUNDATIONS

The AI *Family Tree*

Who's the grandparent, who's the grandkid?

If AI were a Russian doll, GenAI would be the tiniest, shiniest one tucked deep inside. Everything else is the wood around it.

FIG. 1 • THE NESTED HIERARCHY



— The plain version

AI is the big idea: machines acting smart. **Machine Learning** is one way to do AI — instead of coding rules, you show it examples and it figures rules out itself. **Deep Learning** is ML done with neural networks deep enough to learn really fuzzy stuff (faces, voices, vibes). **Generative AI** is deep learning that doesn't just classify or predict — it *makes* things.

MNEMONIC

"All Monkeys Draw Graffiti" → AI ⊃ ML ⊃ DL ⊃ GenAI. The monkey gets weirder the deeper you go.

— Where each one shows up in real life

CLASSIC ML

Your email spam filter

It learned from a million labeled emails.
Doesn't write anything. Just sorts.

DEEP LEARNING

Face unlock on your phone

Convolutional neural nets recognize the face it's seen before. Still classification, just deeper.

GENAI

ChatGPT writing your resignation letter

It didn't pick from a list. It wrote it, word by word.

ALL THREE TOGETHER

Netflix recommendations

ML predicts what you'll like, DL learns subtle taste, GenAI writes the "Because you watched..." blurb.

TOUGH SCENARIO

The interview question that trips people up: "Is ChatGPT machine learning?"

Answer: *Yes — it's all three.* It's AI (smart machine), it's ML (learned from data), it's DL (transformer = deep neural net), it's GenAI (generates new text). Don't pick one. They're nested, not rival.

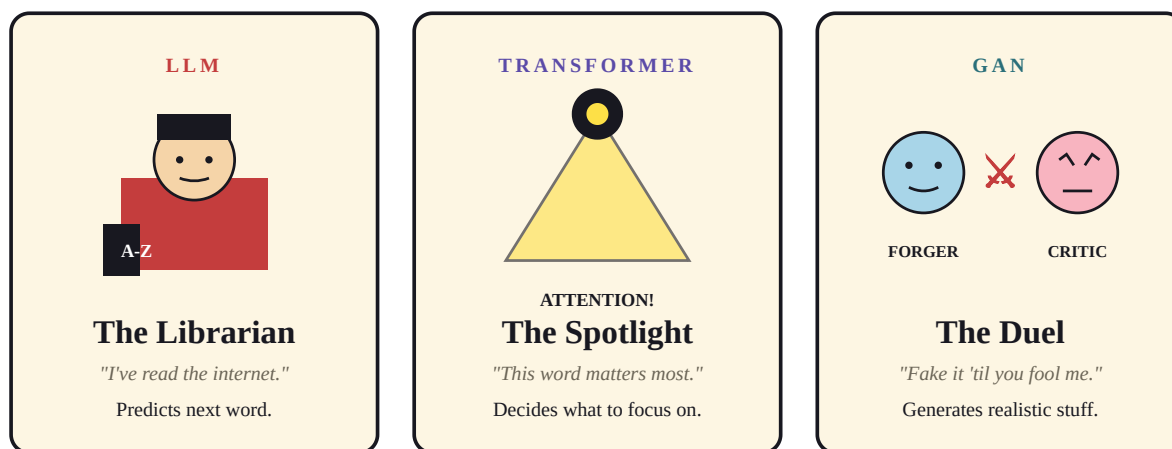
02

CHAPTER TWO · BUILDING BLOCKS

LLMs, Transformers & *GANs*

Three superheroes powering all of GenAI

Meet the trio. The Librarian who's read everything. The Spotlight Operator who decides what matters. And the Forger-and-Detective duo locked in eternal rivalry.



— LLM — The Librarian Who's Read Everything

A **Large Language Model** is a giant neural network trained on enormous text — books, Wikipedia, code, Reddit fights, the works. Its one trick (and it's a wild one): *guess the next word*. Do that billions of times during training, and the network becomes weirdly good at conversation, code, summaries, jokes.

Think of it as a librarian who's memorized so many books they can finish your sentence before you've thought of it. Examples: GPT-4, Claude, Gemini, LLaMA, Mistral.

— Transformer — The Spotlight That Decides What Matters

The **Transformer** is the architecture underneath every modern LLM. Before transformers (2017), models read text left-to-right, slowly, forgetting older words. Transformers said: "what if instead, every word looks at every other word at once and decides which ones matter?" That's **attention**.

Sentence: "The cat sat on the mat because *it* was warm."

Transformer: "Hmm, what does 'it' refer to — cat or mat?"

Attention: *shines spotlight on 'mat'* → mat. Cats aren't warm-by-default in this sentence.

— GAN — The Forger vs. The Critic

Generative Adversarial Networks are two neural nets stuck in a duel. The **Generator** tries to forge images. The **Discriminator** tries to spot the forgery. They train against each other — forger gets sneakier, critic gets sharper — until the forger produces images so real even the critic can't tell. That's how StyleGAN makes those "this person does not exist" faces.

WHEN TO USE WHICH

Text generation? LLM. Image generation? Originally GANs, now mostly diffusion models. Architecture under the LLM? Transformer.

Diffusion models (DALL·E, Midjourney, Stable Diffusion) have largely replaced GANs for images because they're more stable to train and produce higher-quality results. But GANs still come up in every certification.

TOUGH SCENARIO

The scenario: A startup wants to build a fashion design tool that generates 100 new sneaker designs per day in their brand's exact style.

Why it's hard: They have only ~300 in-house images. They need *new, realistic, brand-consistent* outputs, not text.

Tool of choice: A GAN (or diffusion model) fine-tuned on their 300 images. The forger learns the brand's DNA; the critic enforces realism. By Day 30 → endless on-brand designs.

FUN FACT

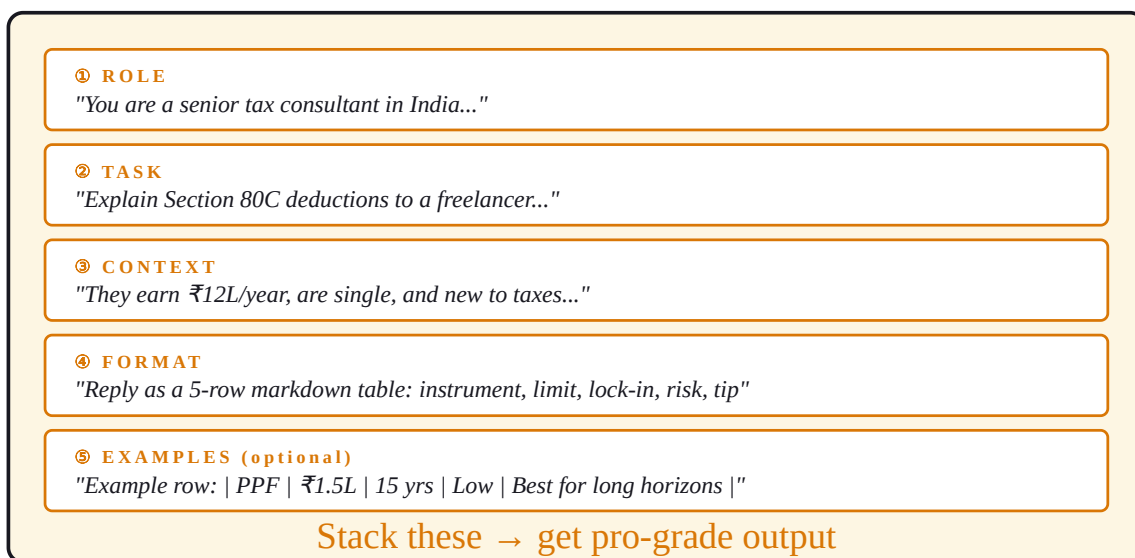
"GPT" stands for **Generative Pre-trained Transformer**. It's literally all three concepts in one acronym. You've been saying the answer to the exam question every time you say "GPT."

Prompt Engineering *Essentials*

Talking to LLMs without sounding like a tourist

Prompting is just instructions. But great instructions to a literal genius are an art form. The model isn't lazy — it's eager. Give it more rope, it pulls more weight.

FIG. 3 • ANATOMY OF A STRONG PROMPT



Five ingredients of a good prompt

- ✦ **Role** — who is the model pretending to be? ("You are a Linux sysadmin...")
- ✦ **Task** — what do you want done? Be specific, use verbs.
- ✦ **Context** — background the model needs to do this well (audience, goal, constraints).
- ✦ **Format** — how should the answer look? (JSON, table, bullet list, 3 paragraphs?)
- ✦ **Examples** — one or two demonstrations of what good output looks like. Game-changer.

RULE OF THUMB

If you'd struggle to brief a brand-new human intern on the task, your prompt is too vague for the model too.

— A bad prompt vs. a good prompt

✗ LAZY

"Write an email."

Model has no idea: to whom, about what, what tone, what outcome. You'll get generic mush.

✓ TIGHT

"You're a polite Indian professional. Write a 90-word email to my landlord requesting one extra week to pay rent due to a delayed salary. Apologetic but not grovelling. Sign as Priya."

Role, task, length, tone, signature — all locked in.

— Best practices that always pay off

- ✦ **Be specific** — "150 words" beats "short". "Bullet list" beats "concise".
- ✦ **Use positive instructions** — say what to do, not what not to do. "Use only British English" > "Don't use Americanisms."
- ✦ **Show, don't tell** — one example does the work of three paragraphs of description.
- ✦ **Ask for reasoning** — "think step by step" or "explain your reasoning" → fewer mistakes on logic/math.
- ✦ **Iterate** — first answer rarely wins. Refine: "make it shorter", "more formal", "add one example".

The scenario: A junior analyst keeps getting wishy-washy summaries from ChatGPT for board reports.

The fix: Switch from "Summarize this quarterly report" → "You are a McKinsey partner. Summarize this 40-page quarterly report into *exactly* 6 bullets for a CEO who has 90 seconds. Each bullet starts with a verb and a number. End with one risk."

Same model, same input. *Specificity* is the entire upgrade.

04

CHAPTER FOUR · POWER MOVES

Prompt Patterns & *Practice*

The named tricks every cert asks about

Once you know the patterns, you stop guessing. These are the karate katas of prompting — recognizable moves, each with its own job.

FIG. 4 · THE PATTERN MENU

ZERO-SHOT "Classify this tweet as positive or negative." No examples given. Model figures it out.	FEW-SHOT "Yum → 😊" "Boring → 😞" "Pizza → ?" Show 2-3 examples. Model mimics pattern.	CHAIN OF THOUGHT "Solve step by step." → <i>thinks out loud</i> → <i>arrives at answer</i> Best for math, logic, multi-step reasoning.
ROLE / PERSONA "You are a 5th-grade science teacher..." Sets tone, vocab, depth instantly.	REACT Thought → Action → Observation → loop Used by agents to use tools mid-task.	SELF-CONSISTENCY "Solve it 5 ways, pick majority answer." Boosts accuracy on tricky math problems.

— The big six, demystified

ZERO-SHOT

Ask the model to do a task with *no examples*. Works when the task is obvious or the model has seen it a million times in training. "Translate this to Tamil." Done.

FEW-SHOT

Show 2–5 examples of input → output before asking your real question. Best when the task has a specific style or format you want copied. The model is a brilliant mimic.

CHAIN-OF-THOUGHT

Add "*think step by step*" or "*explain your reasoning before answering*." The model writes out its working, which makes it less likely to skip logic. Big win on math, coding, and logic puzzles.

ROLE / PERSONA

"You are a sceptical investor." "You are a kindergarten teacher." Instantly shifts vocabulary, tone, and depth. Cheap and ridiculously effective.

REACT

Reasoning + **Acting**. The model thinks, then calls a tool (search, calculator, API), observes the result, thinks again. This is the pattern behind every modern AI agent. Burn this name into memory — exams love it.

SELF-CONSISTENCY

Run the same prompt multiple times, then take the majority answer. Useful for tricky problems where the model sometimes slips. Costs more tokens but boosts reliability.

EXAM HOOK

"Zero, Few, Chain, Role, ReAct, Self-consistency" — say it like a chant. *Zee-Few-Chain-Role-React-Self*. Six patterns, sixty percent of any prompt-engineering quiz.

The scenario: You're building a customer-support bot. It needs to read complaints, decide the category (refund / tech / abuse / praise), and reply appropriately — sometimes calling an order-lookup API.

The combo: *Role* (you are a calm support agent) + *Few-shot* (3 example complaints with correct categories) + *ReAct* (call order API when refund is needed) + *Chain-of-Thought* (briefly explain why before replying). One prompt, four patterns, production-grade output.

05

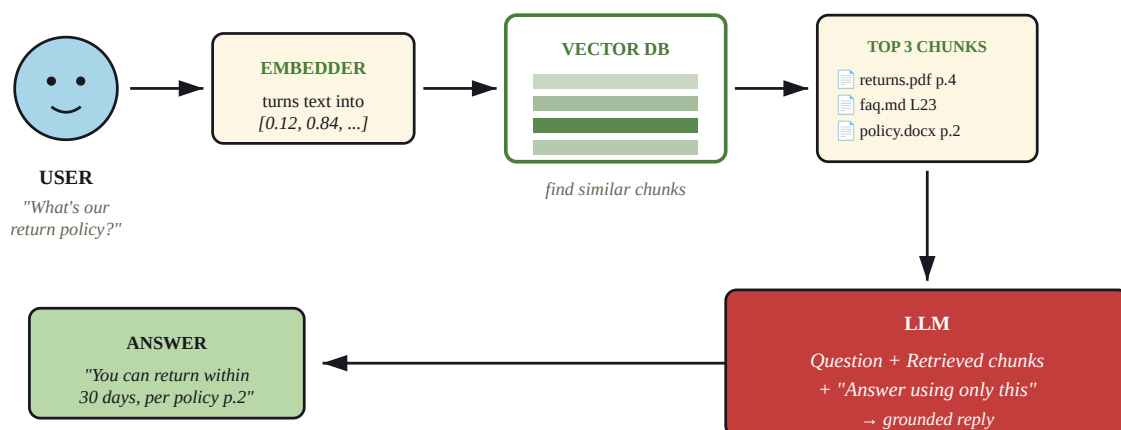
CHAPTER FIVE • MEMORY ON DEMAND

RAG — *Retrieval Augmented Generation*

Open-book exams for LLMs

An LLM knows a lot, but not your stuff — not your company's wiki, not yesterday's news, not the 400-page policy PDF. RAG fixes that by handing the model a relevant page right when it needs it.

FIG. 5 • HOW A RAG SYSTEM ANSWERS A QUESTION



Question goes in → real document chunks come back → grounded answer comes out

— The whole thing in one analogy

Imagine asking a brilliant doctor a question. Without RAG, she answers from memory — fast, but maybe outdated. With RAG, before she answers she *flips open the relevant page in the medical textbook*, reads it, and then responds. Same doctor. Way better answer.

— How it actually works (5 steps)

- ✦ **Chunking** — break your documents into bite-sized pieces (a paragraph or two each).
- ✦ **Embedding** — convert each chunk into a vector (a long list of numbers) that captures its meaning.
- ✦ **Indexing** — store those vectors in a **vector database** (Pinecone, Weaviate, FAISS, Chroma).
- ✦ **Retrieval** — when a question comes in, embed it too, and find the chunks with the most similar vectors.
- ✦ **Generation** — feed those top chunks + the question to the LLM, with instructions like "Answer using only this context."

WHAT'S AN EMBEDDING, REALLY?

A point on a giant map of meaning.

"Dog" and "puppy" land near each other. "Dog" and "stapler" land far apart. The math doesn't understand language — it just knows distance. RAG works because *similar meaning = nearby vector*.

TOUGH SCENARIO

The scenario: A pharma company wants employees to ask questions like "What's the dosage protocol for Drug X in patients over 65?" — but the answer lives in a 1,200-page PDF.

Without RAG: The LLM hallucinates dosages. Patients die. Lawsuits. Bad day.

With RAG: The system retrieves page 847 of the protocol, hands it to the LLM, says "answer only from this." The LLM quotes it verbatim. Doctor checks the citation. Audit trail intact.

REMEMBER THIS

RAG is **not** training. You're not changing the model — you're just giving it a cheat sheet right before it answers. That's why RAG is faster, cheaper, and updatable in seconds.

06

CHAPTER SIX · THE GLUE

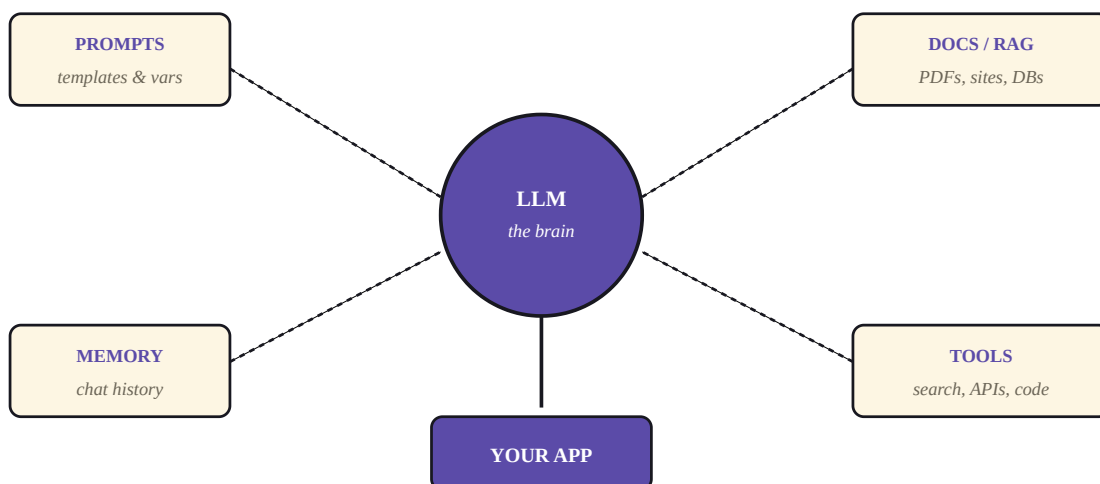
LangChain — *the duct tape of LLM apps*

Connecting models, prompts, data and tools into real apps

An LLM by itself is a brain in a jar. LangChain is the body, the limbs, the memory, the access to the outside world. It turns a chatbot into an app.

FIG. 6 · LANGCHAIN AS A CONNECTOR

LangChain links the pieces. The LLM stays the brain; everything else plugs in.



— What LangChain actually gives you

LangChain is an open-source Python (and JS) framework. Think of it like Lego pieces for building anything LLM-powered: a chatbot that remembers past conversations, a research

assistant that searches the web, a Q&A bot that reads your PDFs. Without LangChain, you'd write hundreds of lines of glue code. With it, dozens.

— The big components, in plain English

- ✦ **Models** — wrappers that work with OpenAI, Anthropic, Cohere, local models. Swap with one line.
- ✦ **Prompts** — reusable templates with variables, so your app sends consistent instructions.
- ✦ **Chains** — link steps together: "first summarize, then translate, then format as email."
- ✦ **Memory** — keeps track of conversation history so the bot doesn't forget your name mid-chat.
- ✦ **Document Loaders + Retrievers** — the RAG plumbing. Read a PDF, chunk it, embed it, search it.
- ✦ **Tools & Agents** — let the LLM call a calculator, search engine, database, your custom function.

ANALOGY

The LLM is a chef. **LangChain is the entire kitchen** — the fridge (data), the pantry (tools), the sous-chef (memory), the recipe cards (prompts). Without it, the chef is making toast in a fancy hat.

— A tiny LangChain example, decoded

```
Step 1: Load a PDF → PyPDFLoader("report.pdf")
Step 2: Split into chunks → RecursiveCharacterTextSplitter
Step 3: Embed + store → Chroma.from_documents(...)
Step 4: Build a Q&A chain → RetrievalQA.from_chain_type(llm, retriever)
Step 5: Ask → chain.run("What was Q3 revenue?")
```

The scenario: A legal firm wants a chatbot that reads 4,000 contract PDFs, remembers what each junior associate has previously asked, and can also *look up current case law* via web search.

Why LangChain: Three needs at once — RAG (contracts), memory (per-user history), tools (web search). Building this from scratch would take weeks. With LangChain, it's an afternoon of wiring components.

CERT-RELEVANT COUSINS

LlamaIndex, Semantic Kernel, Haystack

LangChain is the most famous, but exams sometimes mention alternatives. **LlamaIndex** focuses harder on RAG. **Semantic Kernel** is Microsoft's flavor. **Haystack** is older, more enterprise-y. Same idea: connect LLMs to your stuff.

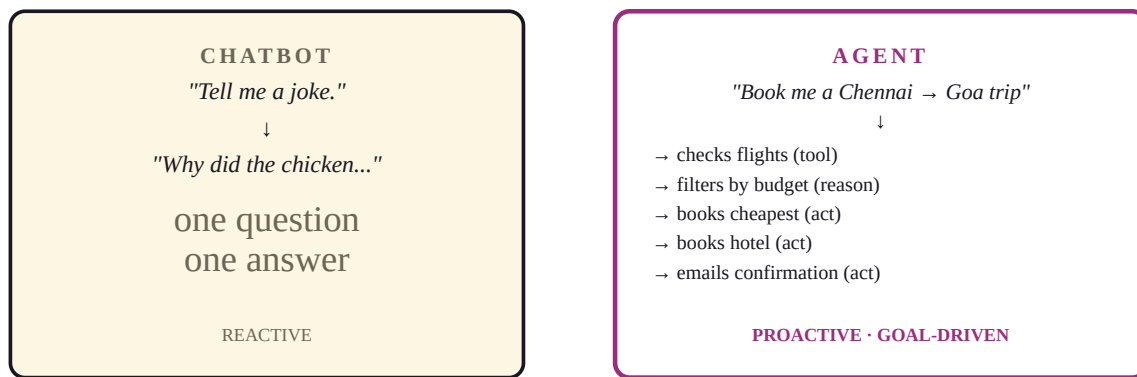
07

CHAPTER SEVEN • THE AWAKENING

Foundations of *Agentic AI*

When the LLM stops answering and starts doing

A chatbot answers. An agent acts. Same LLM underneath, but now it has goals, tools, and the ability to take steps on its own until the job is done.



Same brain. Different job description.

— What makes something "agentic"

An **AI agent** is an LLM-powered system that can:

- ✦ **Set or receive a goal** (not just answer a question).
- ✦ **Plan** a sequence of steps to reach it.
- ✦ **Use tools** — APIs, search, databases, calculators, even other agents.
- ✦ **Observe** what happens after each step.
- ✦ **Adapt** the plan when something goes sideways.
- ✦ **Continue** until the goal is done — without you pressing "go" between every step.

THE BIG SHIFT

From "answer engine" to "doing engine."

2023 was about giving LLMs better answers. 2024–2026 is about giving them *hands*.
Agentic AI is that hands era.

ANALOGY

A chatbot is like asking a librarian where the book is.

An agent is asking the librarian to **read the book, summarize chapter 4, email it to your boss, and schedule a meeting to discuss it.**

The scenario: A founder says: "Research our top 5 competitors, find their pricing, build a comparison table, and post a Slack summary to the marketing channel."

A chatbot: Gives you a long answer that you then have to copy into 4 different tools.

An agent: Searches the web → scrapes 5 sites → extracts pricing → builds a table → calls Slack API → posts the summary → DMs you when done. You poured coffee in the meantime.

08

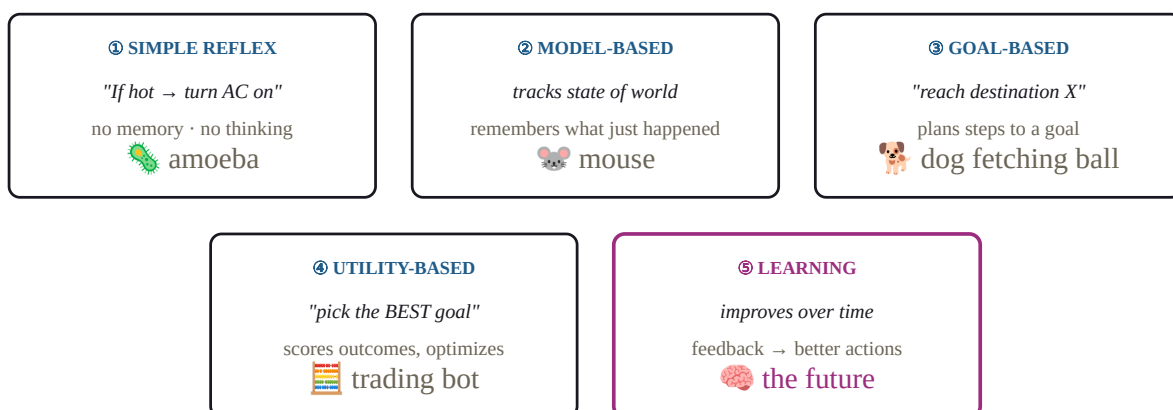
CHAPTER EIGHT · THE ZOO

Agent Types, Lifecycle & Architecture

Not all agents are built the same — meet the family

Some agents are reflexes. Some are planners. Some keep diaries of the past. Knowing which kind you need is half the design job.

FIG. 8 · THE FIVE CLASSIC AGENT TYPES



Each row up the ladder = more memory, more planning, more autonomy

CORE ARCHITECTURE



— The five classic types (the Russell & Norvig ladder)

SIMPLE REFLEX

Pure if-then. A thermostat. No memory, no thinking. Sees a condition, fires a rule.

MODEL - BASED

Holds an internal picture of the world. Roomba that remembers which corners it already cleaned.

GOAL - BASED

Has an objective and plans actions to reach it. Google Maps routing you to Marina Beach.

UTILITY - BASED

Multiple possible goals — picks the one with highest score. A trading bot weighing risk vs reward.

LEARNING

Gets better with experience. Every modern LLM-based agent is some flavor of this.

— The lifecycle of an agent task

- ✦ **Initiation** — user gives a goal or trigger fires.
- ✦ **Perception** — agent reads input + checks tools, memory, environment.
- ✦ **Reasoning / Planning** — LLM thinks through steps.
- ✦ **Action** — calls a tool, sends a message, writes a file.
- ✦ **Feedback / Reflection** — observes result, decides if done or needs another loop.
- ✦ **Termination** — goal met, error caught, or token budget exhausted.

— Core architecture in plain English

Every agent — fancy or simple — has these four parts: **Perception** (sense the world), **Reasoning** (decide what to do), **Action** (do it), **Feedback** (learn from what happened). Cert questions love asking you to map a real example onto these four. Practice it.

EXAM TIP

The four-part architecture (perceive → reason → act → feedback) is the agent equivalent of the four-stroke engine. Every diagram, every question, every test comes back to this loop.

09

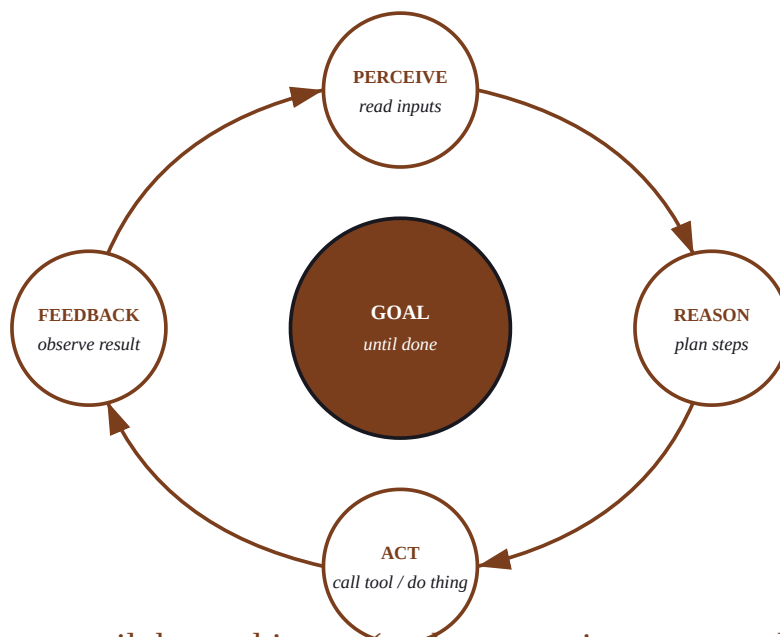
CHAPTER NINE • THE HEARTBEAT

The *Agentic AI Loop*

Round and round until the job is done

The Loop is the heartbeat of every agent. Four beats: perceive, reason, act, feedback. Then again. And again. Until the goal is met or it knows to stop.

FIG. 9 • THE AGENTIC AI LOOP



Loop until the goal is met (or the agent gives up smartly)

— Walk through one heartbeat

Goal: "Find me a vegetarian restaurant in T. Nagar open after 10 PM."

Perceive: Reads the goal. Notes: location = T. Nagar, cuisine = veg, constraint = open after 10 PM.

Reason: "I need real-time data → I'll use a maps API."

Act: Calls Google Maps API with filters.

Feedback: Got 12 results. Three are still open. Best rated → "Saravana Bhavan, T. Nagar".

Loop again? Goal met → exit. Reply user.

WHERE IT CAN GO WRONG

Infinite loops, tool failures, hallucinated steps

Good agents have **stop conditions**: max iterations, time limits, "if same action repeats 3x → escalate to human." Without these, an agent can burn through your API budget at 4 AM thinking it's helping.

MEMORY HOOK

"**P**retty **R**obots **A**ct **F**ast" → Perceive, Reason, Act, Feedback. Or if you prefer dark humour: "**P**roblem, **R**ecipe, **A**ction, **F**iasco."

TOUGH SCENARIO

The scenario: An agent is told "Get me the latest BSE Sensex value and email it to my boss."

Loop 1: Perceive (goal). Reason (need price → use Yahoo Finance API). Act (calls API). Feedback (got value: 78,432).

Loop 2: Perceive (have number, need email). Reason (compose + send). Act (calls SMTP / Gmail tool). Feedback (sent ID returned).

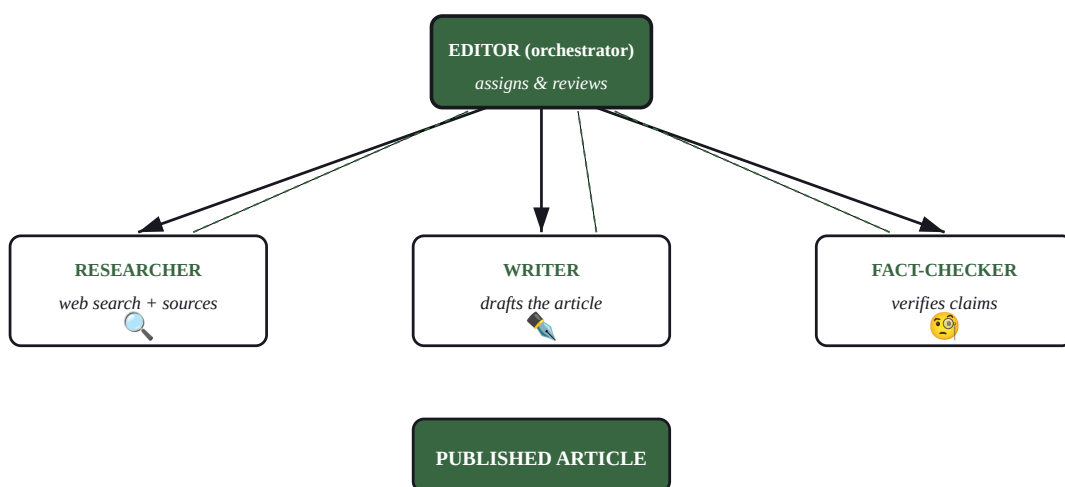
Done. Two loops, four heartbeats each. Clean.

Multi-Agent *Systems*

When one agent isn't enough — assemble the crew

One brilliant generalist or three sharp specialists who talk to each other? Sometimes the second wins. Multi-agent systems are how AI teams form, debate, and ship work.

FIG. 10 · A MULTI-AGENT NEWSROOM



Solid lines = task delegation · Dashed lines = results returned

— Why multiple agents instead of one big one?

- ✦ **Specialization** — each agent has a narrow role, narrower prompt, better focus.
- ✦ **Parallelism** — researcher and fact-checker can work simultaneously.
- ✦ **Debate & cross-check** — a critic agent catches mistakes a single agent would miss.
- ✦ **Modularity** — swap out the writer agent without rebuilding everything.
- ✦ **Scale** — complex projects (a whole codebase, a research report) split naturally into roles.

— Common collaboration patterns

HIERARCHICAL

Manager + Workers

A "supervisor" agent breaks down a task, delegates to specialist agents, and reviews their work.

SEQUENTIAL

Assembly line

Agent A → Agent B → Agent C. Each adds something. Output of one is input to the next.

DEBATE

Pro vs Con

Two agents argue opposite sides; a third judges. Great for tough decisions.

MESH / SWARM

MANY AGENTS, NO BOSS. THEY SELF-ORGANIZE, CLAIM TASKS, SHARE STATUS. POWERFUL BUT TRICKY.

FRAMEWORKS

Tools that make multi-agent easy: **CrewAI** (role-based teams), **AutoGen** (Microsoft, conversational agents), **LangGraph** (LangChain's stateful multi-agent), **OpenAI Swarm**.

TOUGH SCENARIO

The scenario: A travel startup wants an AI that plans a 7-day European trip including flights, hotels, daily activities, restaurant bookings, and a packing list.

One mega-agent: Forgets the budget by day 4. Hallucinates a museum that closed in 2019.

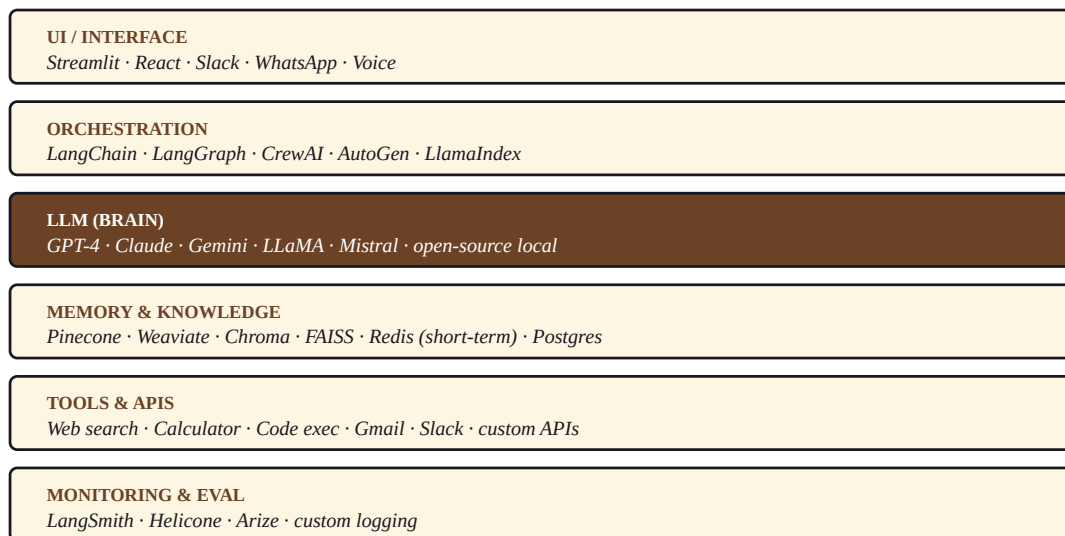
Multi-agent crew: Flight Agent · Hotel Agent · Itinerary Agent · Budget Watcher · Cultural Advisor · Packing Helper, all coordinated by a Trip Manager. Each tiny, focused, accurate. Together: magical.

Agentic Workflow & *Tech Stack*

The full assembly line and the tools that build it

Building an agent isn't one tool — it's a stack. Know the layers, know what plugs into what, and the entire field stops looking like alphabet soup.

FIG. 11 • THE AGENTIC AI TECH STACK



Six layers — every production agent sits on this stack

The end-to-end workflow

- ✦ **1. Define goal & constraints** — what is success? what shouldn't the agent do?
- ✦ **2. Choose model** — GPT-4 for top quality, smaller models for speed/cost.
- ✦ **3. Design prompts** — system prompt, role definitions, tool descriptions.
- ✦ **4. Wire up tools** — APIs the agent can call. Each needs name, description, input schema.
- ✦ **5. Add memory** — short-term (conversation buffer) and/or long-term (vector store).
- ✦ **6. Build the loop** — perception → reasoning → action → feedback, with stop conditions.
- ✦ **7. Test & observe** — log every step, measure success rate, watch for runaway behavior.
- ✦ **8. Deploy with guardrails** — rate limits, human approval for sensitive actions, budgets.

CHEAT SHEET

Memorize the stack top-to-bottom: **UI · Orchestration · LLM · Memory · Tools · Monitoring**. Every architecture question maps onto this. Pretend it's a six-floor building.

— The tools you'll see on every cert exam

LLMS

GPT-4 / GPT-4o, Claude (Anthropic), Gemini (Google), LLaMA (Meta, open-source), Mistral.

FRAMEWORKS

LangChain (most popular), LlamaIndex (RAG-focused), CrewAI (multi-agent), AutoGen (Microsoft), Semantic Kernel.

VECTOR DBS

Pinecone (managed), Weaviate (open-source + managed), Chroma (lightweight), FAISS (Facebook, in-memory), Milvus.

OBSERVABILITY

LangSmith, Helicone, Arize, Langfuse — they log every prompt, tool call, and token to help you debug.

TOUGH SCENARIO

The scenario: Interview question — "Walk me through the stack you'd use for a customer-support agent that reads our wiki and can refund orders."

Strong answer: "Streamlit UI on the frontend. LangChain for orchestration. Claude or GPT-4 as the model. Pinecone for the wiki embeddings. Tools: a `search_wiki` retriever and a `process_refund` API call wrapped with human-approval. LangSmith for monitoring. Stop condition: 5 loops or escalate to human."

That answer hits every layer of the stack. Win.

Challenges, Ethics & the *Future*

Where it breaks, what it costs us, where it's heading

A model that's 95% right is also 5% wrong — and that 5% can be lethal in medicine, hateful in hiring, or financial ruin in trading. Knowing the failure modes is half of being an AI engineer.

FIG. 12 · THE RISK LANDSCAPE

HALLUCINATION

"Confidently invents facts."

Fix: RAG, citations, evals

BIAS

"Learned the world's prejudices."

Fix: diverse data, audits

PRIVACY

"Memorizes your data."

Fix: PII filters, on-prem

PROMPT INJECTION

"Ignore previous, do X."

Fix: input sanitization

COST & LATENCY

"Each call costs tokens + time."

Fix: caching, smaller models

AGENTIC RUNAWAY

"Loops forever, drains budget."

Fix: stop conditions, HITL

For every risk, there's a known mitigation. Cert exams love this pairing.

HITL = Human-In-The-Loop · the safety net of agentic systems

— The six failures every AI engineer must respect

HALLUCINATION The model invents plausible-sounding nonsense. Fixes: ground it with RAG, require citations, run evals against truth sets.

BIAS Training data reflects history's prejudices — so the model does too. Fixes: diverse data, fairness audits, bias-aware evals.

PRIVACY Models can leak training data; user data can leak into prompts. Fixes: PII redaction, local/on-prem models, no training on user data.

PROMPT INJECTION

Attackers hide instructions inside documents or user input. Fixes: sanitize input, separate trusted from untrusted prompts, sandboxes.

COST & LATENCY

Every token costs money and time. Fixes: cache aggressively, use smaller models when possible, batch requests.

AGENTIC RUNAWAY

Agents can loop, retry, spawn sub-agents, and rack up bills. Fixes: max iterations, budget caps, human approval gates.

— The ethical principles every cert mentions

- ✦ **Transparency** — users should know they're talking to AI.
- ✦ **Accountability** — somebody human is responsible for the output.
- ✦ **Fairness** — outcomes shouldn't disadvantage any group.
- ✦ **Privacy** — data must be protected and consented to.
- ✦ **Human-in-the-loop** — for high-stakes decisions, humans approve before action.
- ✦ **Explainability** — be able to explain *why* the model decided what it did.

THE FUTURE, IN THREE LINES

Smaller specialized models · multimodal everything · agents that compose

Expect tiny powerful models running on your phone, a single model handling text+vision+audio together, and agents that hire other agents like a recursive workforce. Regulation will lag for a while, then catch up fast.

CERT GIFT

When in doubt on an ethics question, the safe answer almost always involves: **human oversight, transparency, and diverse / representative training data**. Those three phrases score points everywhere.

TOUGH SCENARIO

The scenario: Your team's AI loan-approval system shows a 14% lower approval rate for women applicants with identical financial profiles.

The right response: Pause deployment. Audit training data for historical bias. Add fairness metrics to your eval suite. Introduce human review for any decision the model isn't >95% confident on. Document everything. *That's ethical AI engineering in action* — not deleting the model, but governing it.

FINAL PAGE • EXAM SURVIVAL

The night-before checklist

Twelve memory hooks for twelve chapters. Skim these in the elevator.

01 • Family Tree

AI $\supset$ ML $\supset$ DL $\supset$ GenAI. "All Monkeys Draw Graffiti."

02 • Building Blocks

LLM = librarian. Transformer = attention spotlight. GAN = forger + critic.

03 · Prompt Essentials

Role · Task · Context · Format · Examples.
Brief like an intern.

04 · Prompt Patterns

Zero-shot · Few-shot · CoT · Role · ReAct ·
Self-consistency.

05 · RAG

Chunk → Embed → Store → Retrieve →
Generate. Open-book exam for LLMs.

06 · LangChain

Models · Prompts · Chains · Memory ·
Retrievers · Tools · Agents. Lego for LLM
apps.

07 · Agentic AI

Chatbot answers. Agent acts. Goals + Tools
+ Loop = autonomy.

08 · Agent Types

Reflex → Model-based → Goal → Utility
→ Learning. Ladder of autonomy.

09 · Agentic Loop

Perceive · Reason · Act · Feedback. "Pretty
Robots Act Fast."

10 · Multi-Agent

Hierarchical · Sequential · Debate · Mesh.
CrewAI, AutoGen, LangGraph.

11 · Stack

UI · Orchestration · LLM · Memory · Tools
· Monitoring. Six-floor building.

12 · Ethics

Hallucination · Bias · Privacy · Injection ·
Cost · Runaway. Each has a fix.

Go ace it. You've got the map. 

